

- This exam has 5 questions for a total of 60 marks.
- Warning: CMI's academic policy regarding cheating applies to this exam.
- You can use results seen in class without reproving them. You may use basic data structures like arrays, heaps, binary search trees, linked lists. However, if you use any other data structure, you have to describe its structure as well as justify its update and query time.
- Keep your answers precise. A short, clear, precise answer is always preferred over a long, ambiguous answer.
- Whenever you are asked to design an algorithm, you must make it as efficient as possible, also prove its correctness and analyze the running time. The credit will depend on the running time of your algorithm as well, apart from its correctness.

1. We have seen in class that any comparison-based sorting algorithm needs $\Omega(n \log n)$ time in the worst case. Can there be a comparison-based sorting algorithm that takes $O(n)$ time on:

(a) at least half of all possible inputs? *Note that there are $n!$ possible inputs, one corresponding to each permutation of the sorted order.* [7]

(b) at least a $\frac{1}{n}$ fraction of all possible inputs? [7]

Your answers should hold for large values of n . Justify each answer.

2. The input is a directed acyclic graph $G = (V, E)$, with two designated vertices s, t . The goal is to count the number of paths from s to t in G . Give a linear-time algorithm for this problem. You can assume that addition of two numbers takes $O(1)$ time. There is no credit for an exponential-time algorithm. [10]

3. Given an n -sided convex (but not necessarily regular) polygon in terms of the cartesian coordinates of its vertices, the goal is to triangulate the polygon into $n - 2$ triangles such that the sum of perimeter of the triangles is as small as possible. [10]

Hint: Consider a fixed side of the polygon as your starting point.

4. There is a set of requests $S = \{[x_i, y_i] \mid i \in [n]\}$ for lecture halls. Each request $[x_i, y_i]$ means there is a class to be scheduled from time x_i to time y_i . Of course, overlapping classes have to be scheduled in different lecture halls. The goal is to assign a lecture hall for each request, minimizing the total number of lecture halls used.

Consider the following proposed algorithms:

(a) Assign as many non-overlapping requests as possible to Lecture Hall 1, then among the remaining requests, assign as many as possible to Lecture Hall 2, and so on. [7]

(b) Process requests in increasing order of start times i.e. $i_1, \dots, i_n$ where $x_{i_1} \leq x_{i_2} \leq \dots \leq x_{i_n}$. For a request $[x_i, y_i]$, examine the lecture halls already assigned to one or more of previous requests. If any of them can be assigned to $[x_i, y_i]$ without overlapping with previous requests, then do so. Otherwise assign a new lecture hall to $[x_i, y_i]$. [7]

Does either algorithm (or do both algorithms) give an optimal solution? Justify your answer by giving a counter-example for a *NO* answer and a correctness proof and complexity analysis for a *YES* answer.

5. A company is building a network that has n servers, which need to send messages to each other. Some pairs of servers have already been connected by *point-to-point one-way* links. A server A can also send messages to another server B via other servers $C_1, \dots, C_k$ provided there is a link from A to C_1 , from C_i to C_{i+1} for each $i \in [k-1]$, and from C_k to B . The goal is to add a minimum number of links to this network so that any server can send messages to any other server. Any link only provides a one-way connection. [12]

Give a linear-time algorithm for this problem.